

Метрики оценки моделей и кросс-валидация

Часть 1. Метрики регрессии

После обучения модели нужно понять, насколько хорошо она предсказывает. Для этого служат **метрики** — числа, измеряющие расхождение между реальными значениями y и предсказанными $\hat{y}$. Они позволяют оценить качество модели, сравнивать модели между собой и замечать переобучение. Для регрессии (предсказание непрерывной величины) применяют четыре основные метрики.

Для каждой метрики ниже указаны: полное название, формула, назначение, интерпретация результата, преимущества и недостатки, примеры применения.

Обозначения: n — число объектов; y_i — реальное значение; $\hat{y}_i$ — предсказание; $\bar{y}$ — среднее реальных значений.

MAE — Mean Absolute Error (средняя абсолютная ошибка)

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Назначение: средняя величина ошибки в тех же единицах, что и целевая переменная.

Интерпретация: «в среднем модель ошибается на MAE единиц»; чем меньше — тем лучше, 0 — идеально.

Применение: прогноз спроса, времени доставки, температуры — когда нужна понятная ошибка и в данных есть выбросы.

Плюсы: проста и интуитивна; в исходных единицах; устойчива к выбросам.

Минусы: все ошибки равнозначны (не выделяет крупные); модуль не дифференцируем в нуле.

MSE — Mean Squared Error (средняя квадратичная ошибка)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Назначение: средний квадрат ошибок; сильно штрафует крупные промахи.

Интерпретация: меньше — лучше, но значение в квадрате единиц, «на глаз» не читается.

Применение: функция потерь при обучении (метод наименьших квадратов); задачи, где крупные ошибки опасны.

Плюсы: гладкая (удобна для оптимизации); делает акцент на больших ошибках.

Минусы: единицы измерения в квадрате (трудно интерпретировать); чувствительна к выбросам.

RMSE — Root Mean Squared Error (корень из средней квадратичной ошибки)

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Назначение: корень из MSE — ошибка в исходных единицах с акцентом на крупные промахи.

Интерпретация: «типичная» ошибка в единицах y ; меньше — лучше; всегда $\geq$ MAE.

Применение: прогноз цен (недвижимость), Kaggle-соревнования, отчёты для бизнеса.

Плюсы: интерпретируема как MAE, но штрафует большие ошибки как MSE.

Минусы: по-прежнему чувствительна к выбросам (наследует от MSE).

R² Score — Coefficient of Determination (коэффициент детерминации)

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Назначение: доля дисперсии целевой переменной, объяснённая моделью.

Интерпретация: 1 — идеально; 0 — не лучше среднего; < 0 — хуже среднего. «Модель объяснила X% разброса».

Применение: быстрая оценка качества и сравнение моделей в отчётах.

Плюсы: безразмерная (0...1) — удобно сравнивать модели на разных данных.

Минусы: растёт от добавления лишних признаков ($\rightarrow$ Adjusted R²); не показывает величину ошибки в единицах.

Сводная таблица метрик

Метрика	Единицы	Реакция на выбросы	Когда применять
MAE	как у y	низкая	нужна понятная средняя ошибка, есть выбросы
MSE	квадрат единиц	высокая	обучение модели; крупные ошибки критичны
RMSE	как у y	высокая	интерпретируемая оценка с акцентом на промахи
R ²	безразмерная	средняя	сравнить модели / «сколько % объяснено»

Часть 2. Кросс-валидация

Кросс-валидация (cross-validation, CV) — способ оценить модель надёжнее, чем по одному разбиению. Данные несколько раз делят на обучающую и проверочную части так, чтобы каждый объект побывал в проверке, и усредняют метрику. Это даёт более устойчивую и честную оценку качества.

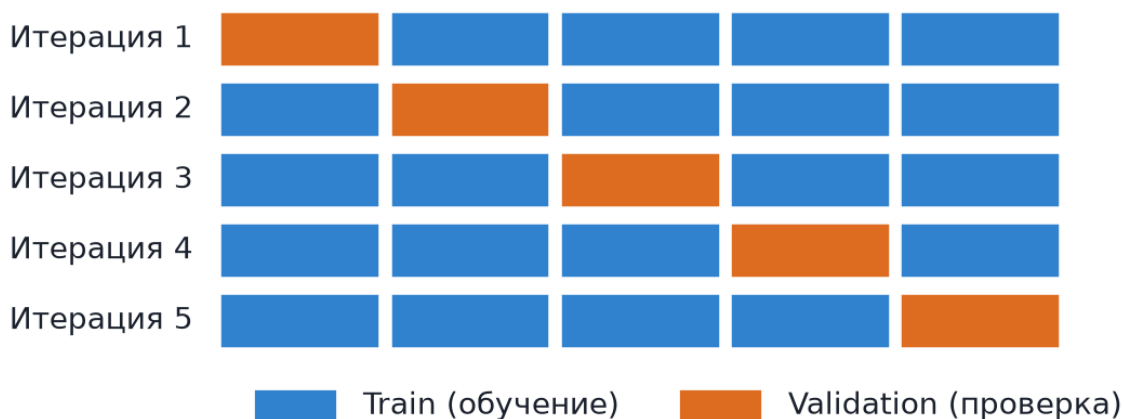
Почему обычного Train/Test иногда недостаточно

- результат зависит от того, какие объекты случайно попали в тест — можно поймать «удачный» или «неудачный» сплит;
- на маленьких данных одна оценка ненадёжна (большой разброс между запусками);
- подбирая гиперпараметры по одному тесту, мы незаметно «подгоняемся» под него.

Hold-out (один Train/Test сплит) проще и быстрее, но даёт лишь одну оценку. Кросс-валидация усредняет оценку по нескольким разбиениям и снимает эту случайность.

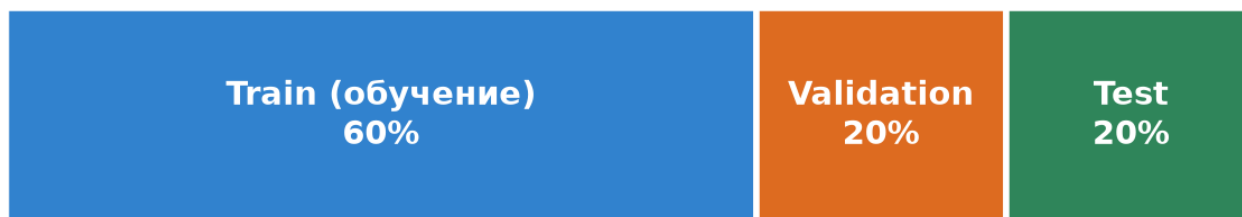
Принцип K-Fold Cross Validation

Данные делят на K равных частей (фолдов). По очереди каждый фолд становится проверочным, остальные K−1 — обучающими. Получаем K оценок метрики и усредняем их (на схеме K = 5):



Train / Validation / Test

Перед кросс-валидацией данные обычно делят на три роли:



- **Train set** (обучающая) — на ней модель подбирает свои параметры (учится).
- **Validation set** (валидационная) — на ней подбирают гиперпараметры и сравнивают модели; при CV её роль по очереди играет каждый фолд.
- **Test set** (тестовая) — отложенная нетронутая выборка для финальной оценки ОДИН раз; её нельзя использовать при подборе.

Как выбирается значение K

- обычно K = 5 или 10 — баланс между надёжностью и временем (учитель использует K = 5);
- больше K → оценка надёжнее, но дольше считать; K = n — крайний случай Leave-One-Out (каждый объект — отдельный фолд);
- меньше K → быстрее, но оценка менее устойчива;
- для несбалансированной классификации берут Stratified K-Fold — он сохраняет доли классов в каждом фолде.

Преимущества и недостатки кросс-валидации

Плюсы: надёжная оценка; каждый объект участвует и в обучении, и в проверке; снижает влияние «удачного» сплита; помогает честно подбирать гиперпараметры.

Минусы: дольше считать (модель обучается K раз); требует аккуратности при утечке данных и на временных рядах (там нужен особый TimeSeriesSplit).

Пример на Python (cross_val_score, KFold)

```
from sklearn.model_selection import cross_val_score, KFold
from sklearn.linear_model import LinearRegression

kf = KFold(n_splits=5, shuffle=True, random_state=42)
model = LinearRegression()
scores = cross_val_score(model, X, y, cv=kf, scoring='r2')

print('R2 по фолдам:', scores)
print('Средний R2:', scores.mean())
```

Вывод

Метрики показывают, **насколько точно** модель, а кросс-валидация — **насколько этой оценке можно доверять**. На практике их используют вместе: метрику (MAE / RMSE / R^2) считают на каждом фолде кросс-валидации и усредняют — так получают устойчивую итоговую оценку модели.